

- (a) Given the following 4-bit image:

0	4	4	4	4	4	6	7
0	4	5	5	5	4	6	7
1	4	5	5	5	4	6	7
1	4	5	5	5	4	6	7
1	4	4	4	4	4	6	7
2	2	8	8	8	10	10	11
2	2	9	9	9	12	13	13
3	3	9	9	9	15	14	14

transform it to a 3-bit image by removing the least most significant bit plane.

Construct a Huffman code on the result and determine the average number of bits/pixel used by the code.

- (b) Now apply Huffman coding to the original image and determine the average number of bits/pixel used by the code.

- (c) Which of the two codes gives the best rate of compression?

11. By hand, apply LZW encoding to the phrases

- (a) HOWNOWBROWNCOW

- (b) REALLYREALCEREALDEAL and decode the results.

12. Experiment with LZW coding of images. Which image has the best compression rate and which image has the worst?

13. Write a program to perform LZW uncompression of an image code, and apply it to the codes from the previous question.

14. Apply JPEG compression to an 8×8 block consisting of (a) all the same value, (b) the left half one value, and the right half another, (c) random values uniformly distributed in the 0–255 range. Compare the length of the code vector in each case, and the results of decompression.

15. Open the image `engineer.png`. Using the JPEG compression commands described above, attempt to compress this image using greater and greater compression rates. What is the largest quantization scale factor for which the image is still recognizable? How many zeros are in the DCT block matrix?

16. Apply the given JPEG Huffman codes to the vector:

-14 -3 -7 0 4 2 0 0 4 3 0 1 0 -1 -1 0 0 0-1 0 -1 E0B

and determine the compression rate.

15 Wavelets

15.1 Waves and Wavelets

We have seen in Chapters 7 and 14 that the discrete Fourier transform and its cousin, the discrete cosine transform, are of enormous use in image processing and image compression. But as we saw with the Fourier transform, this power does not come without cost. In particular, as the Fourier transform is based on the idea of an image being decomposed into periodic sine or cosine functions, we have to assume some sort of periodicity in the image to make the theory fit the practice. This periodicity was obtained by assuming that the image “bent round” to meet itself at the top and bottom, and at the left and right sides.

But this means that we have introduced unnecessary discontinuities in the image, and these may affect the transform and our use of it.

The idea of wavelets is to keep the wave idea, but drop the periodicity. So we may consider a *wavelet* to be a little part of a wave: a wave which is only non-zero in a small region. Figure 15.1 illustrates the idea. Figure 15.1(a) is just the graph of

$$y = \sin(x)$$

for $-10 \leq x \leq 10$, and Figure 15.1(b) is the graph of a scaled version of

$$y = (1 - x^2)e^{-x^2/2}$$

over the same interval. This second function, in the more general form

$$y = \frac{2}{\sqrt{3}\sigma\pi^{1/4}} \left(1 - \frac{x^2}{\sigma^2}\right) e^{-x^2/(2\sigma^2)}$$

is known as the *Mexican hat wavelet*.

Suppose we are given a wavelet. What can we do with it? Well, if

$$f = w(x)$$

is the function that defines our wavelet, we can:

Dilate it by applying a scaling factor to x : $f(2x)$ would “squash” the wavelet; $f(x/2)$ would expand it

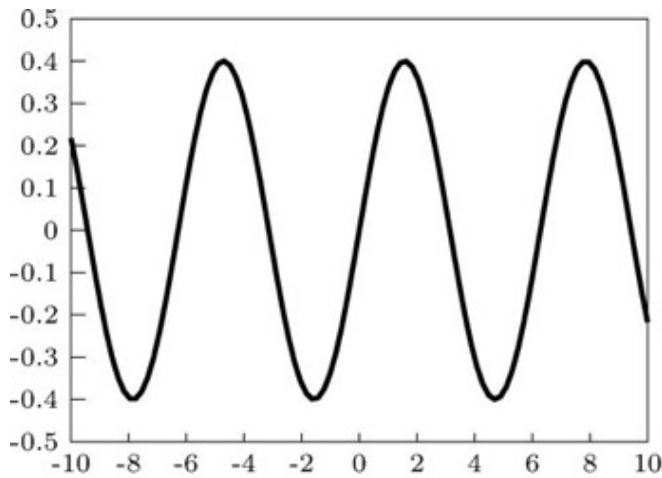
Translate it by adding or subtracting an appropriate value from x : $f(x - 2)$ would shift the wavelet 2 to the right; $f(x + 3)$ would shift the wavelet 3 to the left

Change its height by simply multiplying the function by a constant.

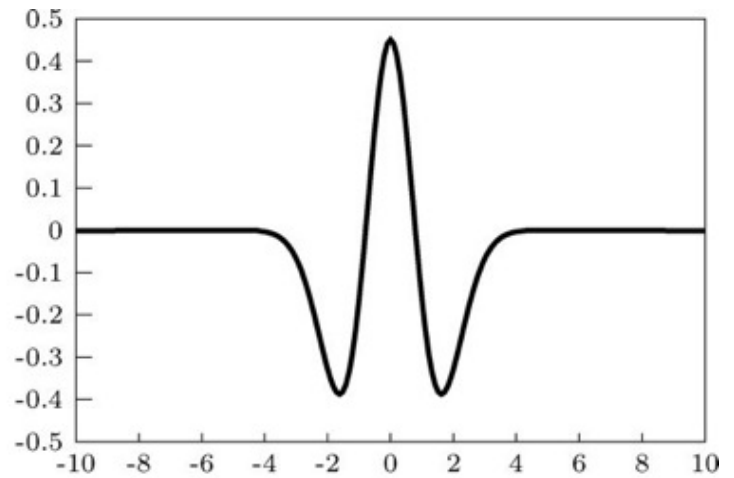
Of course, we can do any or all at once:

$$6f(x/2 - 3), \quad \frac{1}{3}f(16x + 17), \quad f(x/128 - 33.5), \dots$$

Figure 15.1: Comparing a wave and a wavelet



(a) A wave



(b) A wavelet

Let

$$w(x) = \sin(x)e^{-x^2}$$

be the function shown in Figure 15.1(b). Figure 15.2 shows some dilations and shifts of this wavelet.

Given our knowledge of the working of the Fourier transform, it should come as no surprise that given a suitable starting wavelet $w(x)$, we can express any function $f(x)$ as a sum of wavelets of the form

$$aw(bx + c)$$

Moving into two dimensions, we can apply wavelets to images in the same way we applied sine and cosines with the Fourier transform.

Using wavelets has provided a new class of very powerful image processing algorithms: wavelets can be used for noise reduction, edge detection, and compression. The use of wavelets has superseded the use of the DCT for image compression in the JPEG2000 image compression algorithm.

A possible problem with wavelets is that the theory, which has been very well researched, can be approached from many different directions. There is thus no “natural” method of presenting wavelets: it depends on your background, and on the use to which they will be put. Also, much writing on wavelets tends to dive into the theory very quickly. In this chapter, though, we shall just look at the very simplest examples of the use of wavelets and their application to images.

A Simple Wavelet Transform

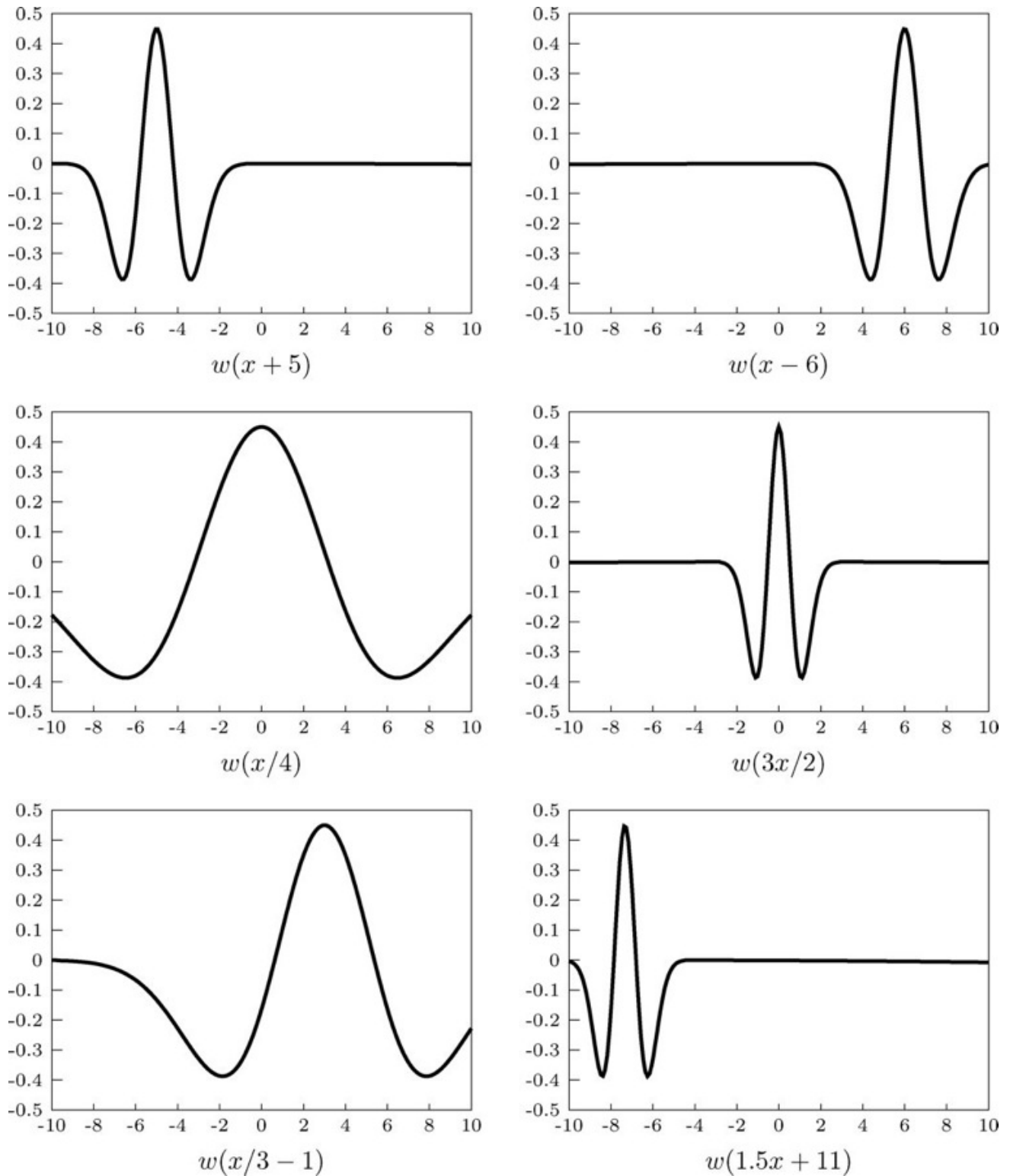
To obtain a feeling for how a wavelet transform works and behaves, we shall look at a very simple example. All wavelet transforms work by taking weighted averages of input values and providing any other necessary information to be able to recover the original input.

For our example, we shall perform just two operations: adding two values and subtracting. For example, suppose we are given two numbers 14 and 22. Their sum and difference are

$$14 + 22 = 36$$

$$14 - 22 = -8$$

Figure 15.2: Shifts and dilations of the wavelet $w(x)$



To recover the original two values from the sum and difference, we take the sum and difference again, and divide by two:

$$(36 - (-8))/2 = 22$$

$$(36 + (-8))/2 = 14$$

In general, if a and b are our two numbers, we have their sum and difference

$$s = a + b$$

$$d = a - b$$

from which the two original numbers can be recovered with

$$\begin{aligned}a &= (s + d)/2 \\ b &= (s - d)/2\end{aligned}$$

The forward operations (adding, subtracting) and the backward operations (adding and dividing by two, subtracting and dividing by two) differ only in the extra division. This function can be extended to any vector with an even number of elements. For example, suppose

$$v = [71, \quad 67, \quad 24, \quad 26, \quad 36, \quad 32, \quad 14, \quad 18].$$

Then define s to be the sum of the elements in pairs:

$$\begin{aligned}s &= [71 + 67, \quad 24 + 26, \quad 36 + 32, \quad 14 + 18] \\ &= [138, \quad 50, \quad 68, \quad 32]\end{aligned}$$

and d to be the difference of pairs:

$$\begin{aligned}d &= [71 - 67, \quad 24 - 26, \quad 36 - 32, \quad 14 - 18] \\ &= [4, \quad -2, \quad 4, \quad -4].\end{aligned}$$

The concatenation of these two vectors:

$$W = [138, \quad 50, \quad 68, \quad 32, \quad 4, \quad -2, \quad 4, \quad -4]$$

is the *discrete wavelet transform at 1 scale* of the original vector.

Recall that in general adding values (such as when using an averaging filter) produces a low pass, blurred output. Conversely, subtracting (such as when using an edge detection filter) produces a high pass output. In general, a wavelet transform will produce a mixture of low and high pass results. These can be separated or combined.

The preceding operation on the vector v could have been performed by a matrix product:

$$\begin{bmatrix} 138 \\ 50 \\ 68 \\ 32 \\ 4 \\ -2 \\ 4 \\ -4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} 71 \\ 67 \\ 24 \\ 26 \\ 36 \\ 32 \\ 14 \\ 18 \end{bmatrix}$$

We can continue by performing this same adding and differencing of the first four elements of the result; this will involve the concatenation of two vectors of two elements each:

$$\begin{aligned}
s_1 &= [138 + 50, \quad 68 + 32] \\
&= [188, \quad 100] \\
d_1 &= [138 - 50, \quad 68 - 32] \\
&= [88, \quad 36]
\end{aligned}$$

Replacing the first four elements of w above with this new s_1 and d_1 produces

$$W_2 = [188, \quad 100, \quad 88, \quad 36, \quad 4, \quad -2, \quad 4, \quad -4]$$

which is the *discrete wavelet transform at 2 scales* of the original vector. We can go one more step, replacing the first two elements of w_2 with their sum and difference:

$$\begin{aligned}
W_3 &= [188 + 100, \quad 188 - 100, \quad 88, \quad 36, \quad 4, \quad -2, \quad 4, \quad -4] \\
&= [288, \quad 88, \quad 88, \quad 36, \quad 4, \quad -2, \quad 4, \quad -4]
\end{aligned}$$

and this is the *discrete wavelet transform at 3 scales* of the original vector.

To recover the original vector, we simply add and subtract, dividing by two each time; first using only the first two elements, then the first four, and finally the lot:

$$\begin{aligned}
&\left[\frac{288 + 88}{2}, \quad \frac{288 - 88}{2}, \quad 88, \quad 36, \quad 4, \quad -2, \quad 4, \quad -4 \right] \\
&= [188, \quad 100, \quad 88, \quad 36, \quad 4, \quad -2, \quad 4, \quad -4]
\end{aligned}$$

$$\begin{aligned}
&\left[\frac{[188, \quad 100] + [88, \quad 36]}{2}, \quad \frac{[188, \quad 100] - [88, \quad 36]}{2}, \quad 4, \quad -2, \quad 4, \quad -4 \right] \\
&= [138, \quad 50, \quad 68, \quad 32, \quad 4, \quad -2, \quad 4, \quad -4]
\end{aligned}$$

$$\begin{aligned}
&\left[\frac{[138, \quad 50, \quad 68, \quad 32] + [4, \quad -2, \quad 4, \quad -4]}{2}, \right. \\
&\quad \left. \frac{[138, \quad 50, \quad 68, \quad 32] - [4, \quad -2, \quad 4, \quad -4]}{2} \right] \\
&= [71, \quad 67, \quad 24, \quad 26, \quad 36, \quad 32, \quad 14, \quad 18]
\end{aligned}$$

At each stage, the sums produce a “lower resolution” version of the original vector. Wavelet transforms produce a mix of lower resolutions of the input, and the extra information required for inversion.

We notice that the differences may be small if the input values are close together. This leads to an idea for compression: we apply a threshold by setting to zero all values in the transform that are less than a predetermined value. Suppose we take a threshold of 0, thus removing all negative values, so that after thresholding W_3 becomes:

$$W'_3 = [288, \quad 88, \quad 88, \quad 36, \quad 4, \quad 0, \quad 4, \quad 0]$$

If we now use this as the starting place for our adding and subtracting, we end up with

$$v' = [71, \quad 67, \quad 25, \quad 25, \quad 36, \quad 32, \quad 16, \quad 16].$$

Notice how close this is to the original vector, in spite of the loss of some information from the transform.

15.2 A Simple Wavelet: The Haar Wavelet

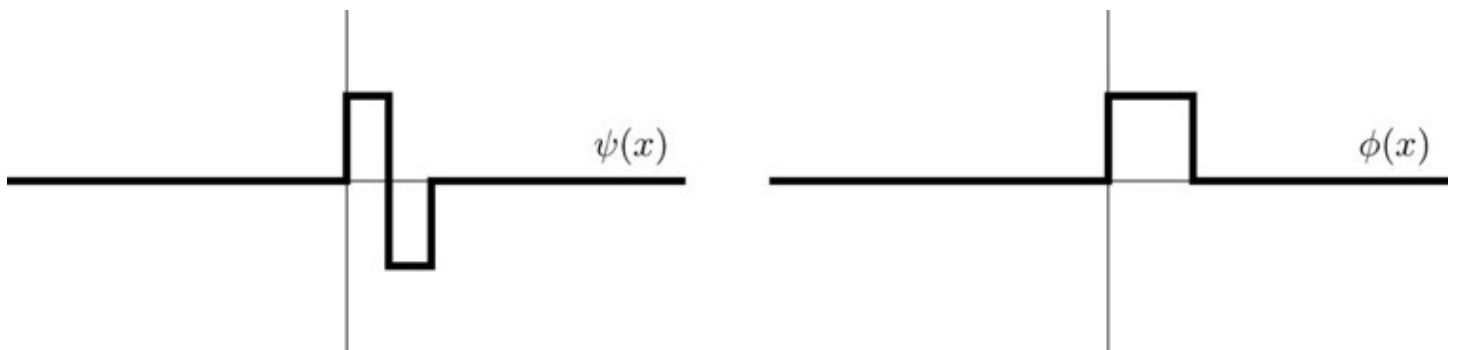
The Haar wavelet has been around for a long time, and has been used with images as the *Haar transform*. Only recently has the Haar transform been viewed as a simple wavelet transform. In fact, the Haar wavelet is the simplest possible wavelet, and for that reason is a good starting place.

The Haar wavelet is defined by the function

$$\psi(x) = \begin{cases} 1 & \text{if } 0 < x < 1/2 \\ -1 & \text{if } 1/2 \leq x < 1 \\ 0 & \text{otherwise} \end{cases}$$

and is shown in Figure 15.3. As with our wavelet function $w(x)$, we can compress and expand this wavelet horizontally or vertically, and shift it.

Figure 15.3: The Haar wavelet and pulse function



Applying the Haar Wavelet

Given the Haar wavelet, what do we do with it? That is, how do we use the Haar wavelet in a wavelet transform? We haven't yet defined what a wavelet transform is, or what it might look like. Without going into too many details, a wavelet transform can be defined in much the same way as the DFT or the DCT: as a sum of function values multiplied by wavelet values. In comparison, for the DFT we multiplied function values by complex exponentials, and in the DCT we multiplied our function values by cosines.

The discrete wavelet transform (DWT) can be written [13] as a sum of input values multiplied by the values from a particular class of functions. And as with the Fourier transform, the discrete wavelet transform can be written as a matrix multiplication. We shall show how this is done.

Notice that the Haar wavelet can be written in terms of the simpler *pulse function*:

$$\phi(x) = \begin{cases} 1 & \text{if } 0 \leq x < 1 \\ 0 & \text{otherwise} \end{cases}$$

using the relation

$$\psi(x) = \phi(2x) - \phi(2x - 1). \quad (15.1)$$

We can see that this is true by noting that $\phi(2x)$ is equal to 1 for $0 \leq x \leq 1/2$, and that $\phi(2x - 1)$ is equal to 1 for $1/2 \leq x \leq 1$.

The pulse function also satisfies the equation

$$\phi\left(\frac{x}{2}\right) = \phi(x) - \phi(x-1). \quad (15.2)$$

In the theory of wavelets, the “starting wavelet,” in this case our function $\psi(x)$, is called the *mother wavelet*, and the corresponding function $\phi(x)$ is called the *scaling function* (and sometimes called a *father wavelet*).

We can also write our scaling expression 15.2 as

$$\phi(x) = \phi(2x) + \phi(2x-1). \quad (15.3)$$

Equations 15.2 and 15.3 are important because they tell us how the wavelets are rescaled at different resolutions. Equation 15.3 is a very important equation: it is called the *dilation equation*, as it relates the scaling function to dilated versions of itself. Generalizations of this equation, as we shall see below, have led to other wavelets than the Haar wavelet. Equation 15.1 is the *wavelet equation* for the Haar wavelet. Note that the dilation and wavelet equations have the same right-hand side, except for a change of sign. These can be generalized to produce

$$\begin{aligned} \phi(x) = & \cdots + h_{-2}\phi(2x+2) + h_{-1}\phi(2x+1) + h_0\phi(2x) + h_1\phi(2x-1) \\ & + h_2\phi(2x-2) + h_3\phi(2x-3) + \cdots \end{aligned} \quad (15.4)$$

$$\begin{aligned} \psi(x) = & \cdots - h_{-2}\phi(2x-3) + h_{-1}\phi(2x-2) - h_0\phi(2x-1) + h_1\phi(2x) \\ & - h_2\phi(2x+1) + h_3\phi(2x+2) - \cdots \end{aligned} \quad (15.5)$$

where the values h_i are called the *filter coefficients* or the *taps* of the wavelet. A wavelet is completely specified by its taps.

We thus have, for the Haar wavelet:

$$\phi(x) = h_0\phi(2x) + h_1\phi(2x-1) \quad (15.6)$$

$$\psi(x) = h_1\phi(2x) - h_0\phi(2x-1) \quad (15.7)$$

where $h_0 = h_1 = 1$. It is actually these h_i values that we use in a calculation of the DWT. We can put them into a DWT matrix:

$$H_{2^n} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & \cdots & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 1 & 1 \\ \hline 1 & -1 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & \cdots & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 1 & -1 \end{bmatrix}$$

If v is the vector we investigated earlier, then $vH_8 = d_1$. We can obtain d_2 and d_3 by multiplying the first values of the vector by the appropriately sized H matrix.

If we compare the results W_2 and W_3 , the transforms at 2 and 3 scales, respectively, with the input, we find that the matrices corresponding to these transforms are:

$$H_{8,2} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & -1 & -1 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \end{bmatrix}$$

and

$$H_{8,3} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & -1 & -1 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & -1 \end{bmatrix}$$

In order to enter such matrices easily, a useful operation is the *Kronecker product* of matrices, denoted $A \otimes B$ and which is defined as the block matrix for which the block at position (i, j) is $a_{ij} B$. For example:

$$\begin{bmatrix} 2 & 3 \\ -1 & 4 \end{bmatrix} \otimes \begin{bmatrix} -2 & 5 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 2 \begin{bmatrix} -2 & 5 \\ 1 & 2 \end{bmatrix} & 3 \begin{bmatrix} -2 & 5 \\ 1 & 2 \end{bmatrix} \\ (-1) \begin{bmatrix} -2 & 5 \\ 1 & 2 \end{bmatrix} & 4 \begin{bmatrix} -2 & 5 \\ 1 & 2 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} -4 & 10 & -6 & 15 \\ 2 & 4 & 3 & 6 \\ 2 & -5 & -8 & 20 \\ -1 & 2 & 4 & 8 \end{bmatrix}$$

With this new product, the matrix H can be defined as

$$H_{2^n} = \begin{bmatrix} I_{2^{n-1}} \otimes [1 & 1] \\ I_{2^{n-1}} \otimes [1 & -1] \end{bmatrix}$$

In MATLAB and Octave, the Kronecker product is implemented with `kron`:

```
>> v = [71 67 24 26 36 32 14 18];
>> H = [kron(eye(4), [1 1]); kron(eye(4), [1 -1])];
>> w = (H*v')'
```

```
w =
```

```
138    50    68    32     4    -2     4    -4
```

MATLAB/Octave

and in Python by `np.kron`:

```

In: from numpy import kron, array, eye, vstack
In : v = array([71, 67, 24, 26, 36, 32, 14, 18])
In : H = vstack([kron(eye(4),[1,1]),kron(eye(4),[1,-1])])
In : w = (H.dot(v.T)).T
In : print w
[ 138.   50.   68.   32.    4.   -2.    4.   -4.]

```

Python

Given

$$H_1 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

as the DWT matrix for 2 variables at 1 scale, a sequence of matrices can be created as:

$$H_{n+1} = \begin{bmatrix} H_n \otimes [1 & 1] \\ I_{2^n} \otimes [1 & -1] \end{bmatrix}$$

Then H_n is the matrix for a DWT of 2^n variables at n scales—the maximum number of scales possible.

Although we have spoken of the DWT formed by adding and subtracting as using the Haar wavelet, this is not precisely true. In order to be a wavelet proper, the DWT matrices must be *orthogonal*; that is

$$HH^T = I.$$

and the H matrices constructed so far are not quite; for example, consider the 4×4 2-scale matrix H . Then:

$$HH^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & -1 & 0 \\ 1 & -1 & 0 & 1 \\ 1 & -1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}.$$

In order to make H orthogonal, each row must be divided by the square root of the diagonal elements in this last matrix:

$$H' = \begin{bmatrix} 1/2 & 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 & -1/2 \\ 1/\sqrt{2} & -1/\sqrt{2} & 0 & 0 \\ 0 & 0 & 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}$$

As an example, here is the 3-scale DWT using the Haar wavelet applied to the original vector. In MATLAB or Octave:

```

>> H = [1 1;1 -1];
>> for i = 1:2, H = [kron(H,[1 1]);kron(eye(size(H)),[1,-1])]; end

```

MATLAB/Octave

So far this produces the 8×8 matrix for a 3-scale transform. Now we must divide the rows by the square roots of the diagonal elements of HH^T :

```
>> D = diag(diag(H*H'))
>> H = sqrt(inv(D))*H
>> format bank
>> w = (H*v')
w =
```

```
101.82    31.11    44.00    18.00     2.83    -1.41     2.83    -2.83
```

MATLAB/Octave

And in Python:

```
In : H = array([[1,1],[1,-1]])
In : for i in range(2):
...:     H = vstack([kron(H,[1,1]),kron(eye(H.shape[0]),[1,-1])])
...:
In : D = H.dot(H.T)
In : H = np.linalg.inv(np.sqrt(D)).dot(H)
In : w = (H.dot(v.T)).T
In : print np.around(w,2)
[ 101.82  31.11  44.    18.    2.83  -1.41  2.83  -2.83]
```

Python

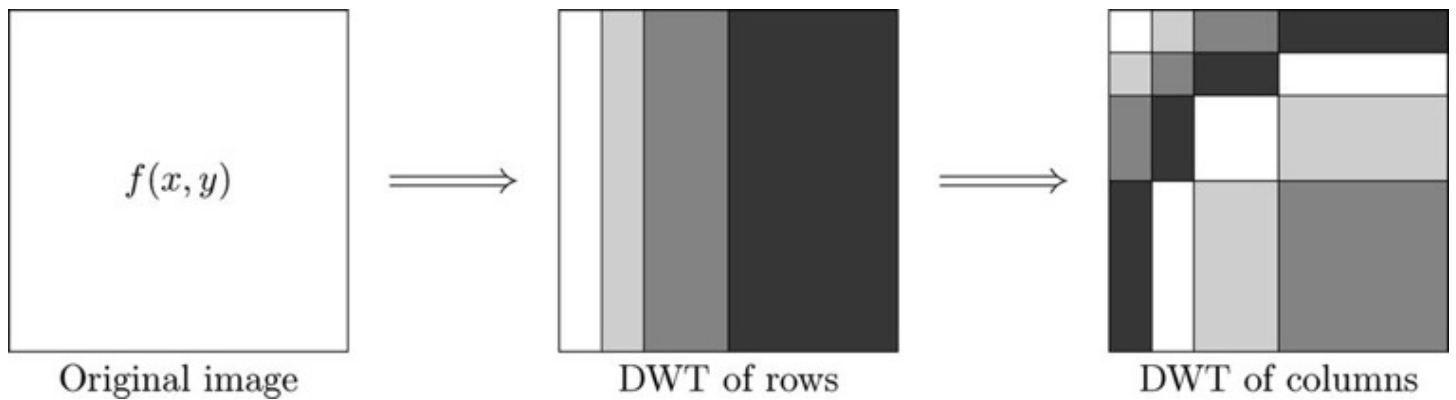
Any wavelet transform can be computed by matrix multiplications. But as we have seen in our discussion of the DFT, this can be very inefficient. Happily many wavelet transforms can be quickly computed by a fast algorithm similar in style to the average/difference method we saw earlier. These methods are called *lifting methods* [24].

If we go back to our averaging and differencing example, we can see that the averaging part of the transform corresponds to low pass filtering, in that we are “coarsening” or “blurring” our input. Similarly, the differencing part of the transform corresponds to a high pass filter, of the sort we looked at in our discussion of edges in Chapter 9. Thus, a wavelet transform contains within it both high and low pass filtering of our input, and we can consider a wavelet transform entirely in terms of filters. This approach will be discussed in Section 15.4.

Two-Dimensional Wavelets

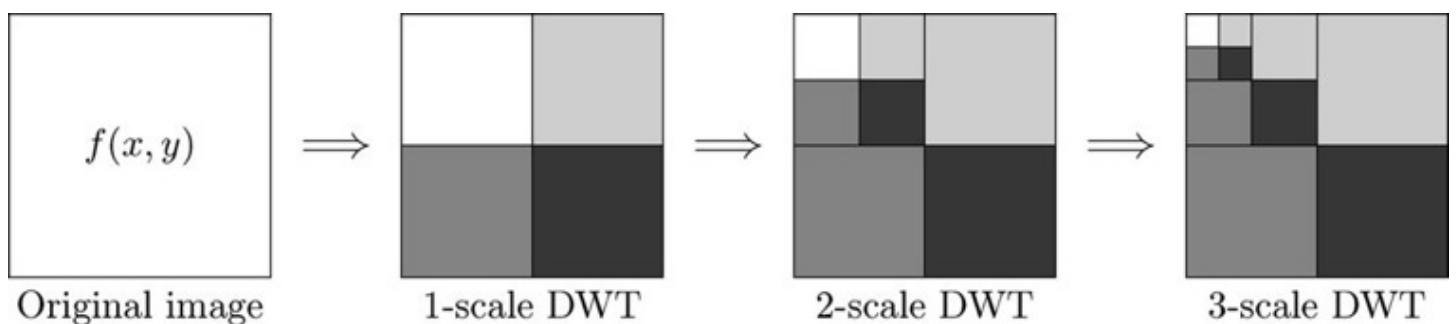
The two-dimensional wavelet transform is *separable*, which means we can apply a one-dimensional wavelet transform to an image in the same way as for the DFT: we apply a one-dimensional DWT to all the columns, and then one-dimensional DWTs to all the rows of the result. This is called the *standard decomposition*, and is illustrated in Figure 15.4.

Figure 15.4: The standard decomposition of the two-dimensional DWT



We can also apply a wavelet transform differently. Suppose we apply a wavelet transform (say, using the Haar wavelet) to an image by columns and then rows, but using our transform at 1 scale only. This will produce a result in four quarters: the top left will be a half-sized version of the image; the other quarters will be high pass filtered images. These quarters will contain horizontal, vertical, and diagonal edges of the image. We then apply a 1-scale DWT to the top left quarter, creating smaller images, and so on. This is called the *nonstandard decomposition*, and is illustrated in Figure 15.5.

Figure 15.5: The non-standard decomposition of the two-dimensional DWT



15.3 Wavelets and Images

Each of MATLAB, Octave, and Python has some wavelet functionality, either built in or available through an added toolbox or package. However, the standard packages all seem to work a little differently. Happily for us, there is an open-source wavelet package with bindings for each of MATLAB, Octave, and Python from Rice University, obtainable from

<http://dsp.rice.edu/software/rice-wavelet-toolbox>

Assuming that the toolbox has been downloaded and installed, we can try it, using the same vector as above:

```
>> [h,g] = daubcwf(2)
h =
    0.70711    0.70711

g =
   -0.70711    0.70711
```

MATLAB/Octave

Here h and g are the low pass and high pass filter coefficients for the forward transform. The `daubcwf` function produces the filter coefficients for a class of wavelets called *Daubechies wavelets*, of which the Haar wavelet is the simplest. Now we can apply the DWT to our vector:

```
>> format bank
>> w = mdwt(v,h,1)
w =

    97.58    35.36    48.08    22.63     2.83    -1.41     2.83    -2.83
```

MATLAB/Octave

This is not quite the same as the first vector W from above, but is off only by a single scaling factor:

```
>> w*sqrt(2)
ans =

    138.00    50.00    68.00    32.00     4.00    -2.00     4.00    -4.00
```

MATLAB/Octave

which is the same result we obtained earlier by adding and subtracting. The transforms at 2 and 3 scales can be given simply by adjusting the final parameter:

```
>> w2 = mdwt(v,h,2)
w2 =

    94.00    50.00    44.00    18.00     2.83    -1.41     2.83    -2.83

>> w3 = mdwt(v,h,3)
w3 =

    101.82    31.11    44.00    18.00     2.83    -1.41     2.83    -2.83
```

MATLAB/Octave

To go backward, use the inverse transform:

```
>> format
>> midwt(w3,h,3)
ans =

    71.000    67.000    24.000    26.000    36.000    32.000    14.000    18.000
```

MATLAB/Octave

Note that we can supply our own filter values:

```
>> h = [1 1]
>> mdwt(v,h,1)
ans =

    138    50    68    32     4    -2     4    -4
>> mdwt(v,h,2)
ans =

    188   100    88    36     4    -2     4    -4
>> mdwt(v,h,3)
ans =

    288    88    88    36     4    -2     4    -4
```

MATLAB/Octave

and these are indeed the values we obtained earlier simply by adding and subtracting.

The Python equivalent commands require the loading of the `rwt` library. Then:

```
In : h,g = rwt.daubcwf(2)
In : print rwt.dwt(v,h,1)[0]
[ 97.5807  35.3553  48.0833  22.6274   2.8284  -1.4142   2.8284  -2.8284]

In : print rwt.dwt(v,h,3)[0]
[ 101.8234   31.1127   44. 18.   2.8284  -1.4142   2.8284  -2.8284]

In : h = np.array([1,1]).astype('float')
In : vw = rwt.dwt(v,h,3)[0]
In : print vw[0]
[ 288.   88.   88.   36.    4.   -2.    4.   -4.]

In : hi = np.array([0.5,0.5]).astype('float')
In : print rwt.idwt(vw[0],hi,3)[0]
[ 71.  67.  24.  26.  36.  32.  14.  18.]
```

Python

Let's try an image. We shall apply the Haar wavelet to an image of size 256×256 , first at one scale. For display, it will be necessary to scale parts of the image for viewing.

```
>> c = imread('cameraman.png');
>> h = daubcwf(2)
>> cw1 = mdwt(double(c),h,1);
```

MATLAB/Octave

Now at this stage some adjustment needs to be done to display the transform. Because the range of values in the transform `cw1` is large:

```
>> [max(cw1(:)),min(cw1(:))]
ans =

    501.00   -209.50
```

MATLAB/Octave

some method will be needed to adjust those values for display. In MATLAB or Octave, since `cw1` is an array of data type `double`, elements outside the range `0.0 – -1.0` will be displayed as black or white. Adjustment can be done in several ways; first by simply using `mat2gray`, second by a log function, as was done for the Fourier transform:

```
>> imshow(cw1)
>> figure, imshow(mat2gray(cw1))
>> cwlog = log(1+abs(cw1))
>> figure, imshow(mat2gray(cwlog))
```

MATLAB/Octave

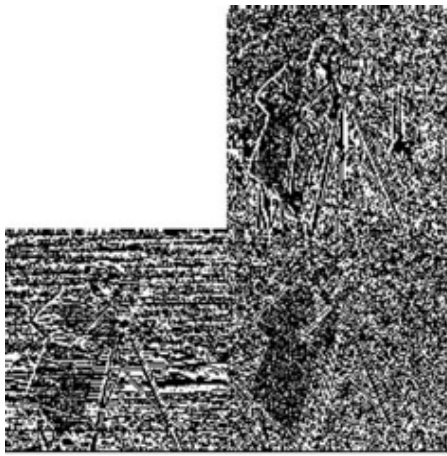
In Python, this is more easily done as images are automatically scaled for display:

```
In : h,g = rwt.daubcwf(2)
In : cw1 = rwt.dwt(castype('float'),h,1)[0]
In : cwlog = np.log(1+abs(cw1))
```

Python

and both `cw1` and `cwlog` can be displayed simply with `io.imshow`. All three images are shown in Figure 15.6. Now consider the same transform, but at 3 scales:

Figure 15.6: Different displays of a 1-scale DWT applied to an image



(a) No adjustment



(b) With mat2gray



(c) With log and mat2gray

```
>> cw3 = mdwt(double(c),h,3);
```

MATLAB/Octave

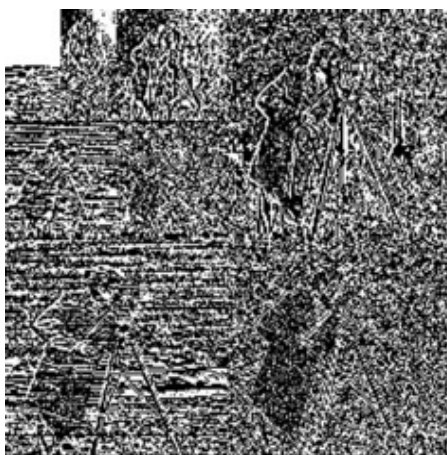
or with

```
In : cw1 = rwt.dwt(castype('float'),h,3)[0]
```

Python

and the result with the same adjustments as above is shown in Figure 15.7.

Figure 15.7: Different displays of a 3-scale DWT applied to an image



(a) No adjustment



(b) With mat2gray



(c) With log and mat2gray

This wavelet transform can be seen to use the non-standard decomposition. To see what a standard decomposition looks like, we can multiply the rows, and then the columns, by the appropriate Haar matrix:


```
>> H = [1 1;1 -1];
>> for i = 1:2, H = [kron(H,[1 1]);kron(eye(size(H)),[1,-1])]; end
```

MATLAB/Octave

or in Python as

```
In : H = np.array([[1,1],[1,-1]])
In : for i in range(2):
...:     H = np.vstack([np.kron(H,[1,1]),np.kron(np.eye(H.shape[0]),[1,-1])
...:                    ])
...:
```

Python

So far this is the 3-scale 8×8 matrix, which now needs to be scaled up to be 256×256 :

```
>> K1 = kron(eye(32),H(1,:));
>> K2 = kron(eye(32),H(2,:));
>> K3 = kron(eye(32),H(3:4,:));
>> K4 = kron(eye(32),H(5:8,:));
>> H = [K1;K2;K3;K4];
```

MATLAB/Octave

or

```
In : K1 = np.kron(np.eye(32),H[0,:])
In : K2 = np.kron(np.eye(32),H[1,:])
In : K3 = np.kron(np.eye(32),H[2:4,:])
In : K4 = np.kron(np.eye(32),H[4:8,:])
In : H = np.vstack([K1,K2,K3,K4])
```

Python

The next step is to scale the rows so that the matrix is orthogonal:

```
>> D = diag(diag(H*H'));
>> H = sqrt(inv(D))*H;
```

MATLAB/Octave

or

```
In : D = H.dot(H.T)
In : np.sqrt(np.linalg.inv(D)).dot(H)
```

Python

This last H is the matrix we want, which implements a Haar 3-scale wavelet transform on vectors of length 256. It can be applied to the image:

```
>> cw = (H*(H*double(c)))')';
```

MATLAB/Octave

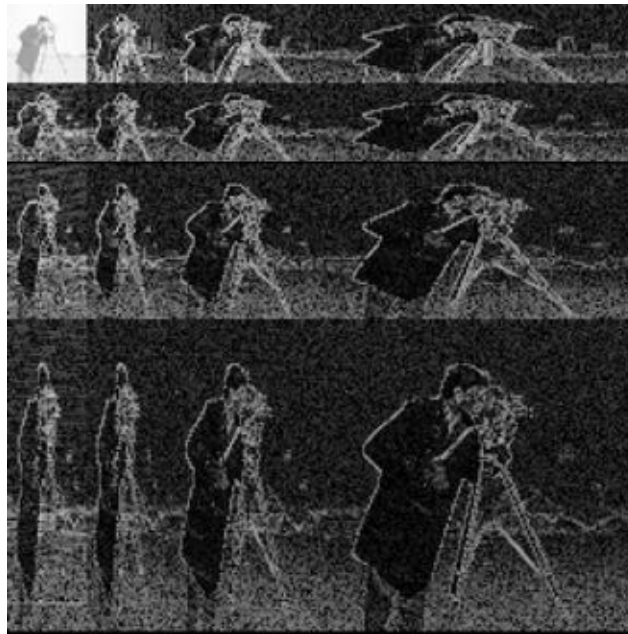
or with

```
In : cw = H.dot(H.dot(c.astype('float'))).T.T
```

Python

and displayed with the log scaling, as shown in Figure 15.8. With the log scaling, it is clear that in the standard decomposition, the filtered images in the transform are squashed, rather than all retaining their shape as in the non-standard decomposition.

Figure 15.8: A 3-scale standard decomposition DWT applied to an image



15.4 The Daubechies Wavelets

We have seen that the dilation and wavelet equations for the Haar wavelet are particular examples of the general equations 15.4 and 15.5. However, it is not at all obvious that there are any other solutions. One of the many contributions of Ingrid Daubechies to the theory of wavelets was to define an entire class of wavelets which may be considered solutions to these equations.

The Daubechies-4 wavelet has a scaling function $\phi(x)$ and wavelet function $\psi(x)$ which satisfy the equations:

$$\phi(x) = h_0\phi(2x) + h_1\phi(2x - 1) + h_2\phi(2x - 2) + h_3\phi(2x - 3) \quad (15.8)$$

$$\psi(x) = h_0\phi(2x - 1) - h_1\phi(2x) + h_2\phi(2x + 1) - h_3\phi(2x + 2) \quad (15.9)$$

where the values of the filter coefficients are:

$$\begin{aligned} h_0 &= \frac{1 + \sqrt{3}}{4\sqrt{2}} \approx 0.48296 \\ h_1 &= \frac{3 + \sqrt{3}}{4\sqrt{2}} \approx 0.83652 \\ h_2 &= \frac{3 - \sqrt{3}}{4\sqrt{2}} \approx 0.22414 \\ h_3 &= \frac{1 - \sqrt{3}}{4\sqrt{2}} \approx -0.12941 \end{aligned}$$

These can be obtained with the `daubcwf` function of `rwt`:

```
>> [h,g] = daubcwf(4)
h =
    0.48296    0.83652    0.22414   -0.12941
g =
    0.12941    0.22414   -0.83652    0.48296
```

MATLAB/Octave

The vectors contain the same values, but in different orders and with different signs. As for the Haar wavelet, we can apply the Daubechies-4 wavelet by a matrix multiplication; the matrix for a 1-scale DWT on a vector of length 8 is

$$\begin{bmatrix} h_0 & h_1 & h_2 & h_3 & 0 & 0 & 0 & 0 \\ 0 & 0 & h_0 & h_1 & h_2 & h_3 & 0 & 0 \\ 0 & 0 & 0 & 0 & h_0 & h_1 & h_2 & h_3 \\ h_2 & h_3 & 0 & 0 & 0 & 0 & h_0 & h_1 \\ h_3 & -h_2 & h_1 & -h_0 & 0 & 0 & 0 & 0 \\ 0 & 0 & h_3 & -h_2 & h_1 & -h_0 & 0 & 0 \\ 0 & 0 & 0 & 0 & h_3 & -h_2 & h_1 & -h_0 \\ h_1 & -h_0 & 0 & 0 & 0 & 0 & h_3 & -h_2 \end{bmatrix}$$

Notice that the filter coefficients overlap between rows, which is not the case for the Haar matrix H_{2^n} . This means that the use of the Daubechies-4 wavelet will have smoother results than using the Haar wavelet. The form of the matrix above is similar to circular convolution with the one-dimensional filters

$$[h_0 \ h_1 \ h_2 \ h_3]$$

and

$$[h_3 \ -h_2 \ h_1 \ -h_0].$$

The discrete wavelet transform can indeed be approached in terms of filtering; the above two filters are then known as the low pass and high pass filters, respectively. Steps for performing a 1-scale wavelet

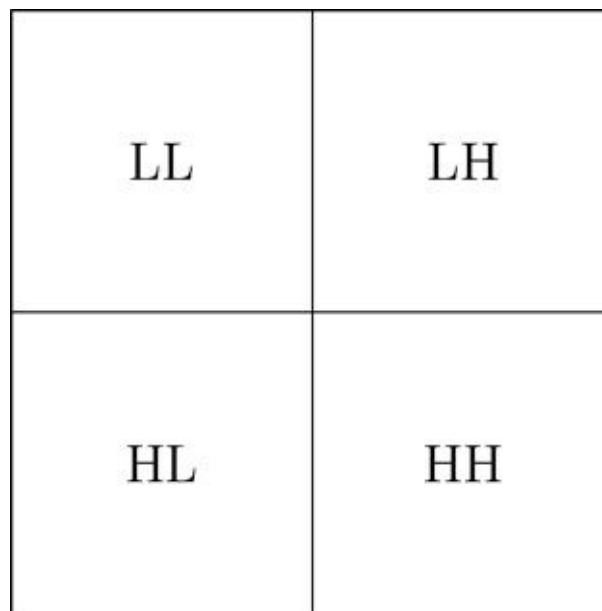
transform are given by Umbaugh [52]:

1. Convolve the image rows with the low pass filter.
2. Convolve the columns of the result of Step 1 with the low pass filter, and rescale this to half its size by subsampling.
3. Convolve the result of Step 1 with the high pass filter, and again subsample to obtain an image of half the size.
4. Convolve the original image rows with the high pass filter.
5. Convolve the columns of the result of Step 4 with the low pass filter, and rescale this to half its size by subsampling.
6. Convolve the result of Step 4 with the high pass filter, and again subsample to obtain an image of half the size.

At the end of these steps there are four images, each half the size of the original. They are:

1. the “low pass/low pass” image (LL); result of Step 2.
2. the “low pass/high pass” image (LH); result of Step 3.
3. the “high pass/low pass” image (HL); result of Step 5.
4. the “high pass/high pass” image (HH); result of Step 6.

Figure 15.9: The 1-scale wavelet transform in terms of filters



These can then be placed into a single image grid as shown in Figure 15.9.

The filter coefficients of a wavelet are such that the transform may be inverted precisely to recover the original image. Using filters, this is done by taking each subimage, zero-interleaving to produce an image of double the size, and convolving with the inverse low pass and high pass filters. Finally, the results of all the filterings are added. For the Daubechies-4 wavelet, the inverse low pass and high pass filters are:

$$[h_2 \ h_1 \ h_0 \ h_3]$$

and

$$\begin{bmatrix} h_3 & -h_0 & h_1 & -h_2 \end{bmatrix}$$

respectively.

A further approach to the discrete wavelet transform may be considered as a generalization to filtering; it is called *lifting*, and is discussed by Jensen and la Cour-Harbo [24]. Lifting starts with a sequence $s_j[n]$, $n = 0 \dots 2^j - 1$ and produces two sequences $s_{j-1}[n]$, $n = 0 \dots 2^{j-1} - 1$ and $d_{j-1}[n]$, $n = 0 \dots 2^{j-1} - 1$, each half the length of the original. The lifting scheme for the Haar wavelet may be described as

$$\begin{aligned} d_{j-1}[n] &= s_j[2n+1] - s_j[2n] \\ s_{j-1}[n] &= s_j[2n] + d_{j-1}[n]/2, \end{aligned}$$

and a lifting scheme for the Daubechies-4 wavelet is:

$$\begin{aligned} s_{j-1}^{(1)}[n] &= s_j[2n] + \sqrt{3}s_j[2n+1] \\ d_{j-1}^{(1)}[n] &= s_j[2n+1] - \frac{1}{4}\sqrt{3}s_{j-1}^{(1)}[n] - \frac{1}{4}(\sqrt{3}-2)s_{j-1}^{(1)}[n-1] \\ s_{j-1}^{(2)} &= s_{j-1}^{(1)}[n] - d_{j-1}^{(1)}[n+1] \\ s_{j-1}[n] &= \frac{\sqrt{3}-1}{\sqrt{2}}s_{j-1}^{(2)}[n] \\ d_{j-1}[n] &= \frac{\sqrt{3}+1}{\sqrt{2}}d_{j-1}^{(1)}[n]. \end{aligned}$$

Both the Haar and Daubechies-4 schemes can be reversed; see [24] for details. A single lifting, as shown above, produces a 1-scale wavelet transform. To produce transforms of higher scales, lifting can be applied to s_{j-1} to produce s_{j-2} and d_{j-2} , and then to s_{j-2} , and so on. In each lifting scheme, the sequence s_{j-1} may be considered the subsampled

low pass version of s_j , and the sequence d_{j-1} may be considered the subsampled high pass version of s_j . Thus, a single lifting scheme provides, in effect, both the low pass and high pass filter results. Applying a lifting to an image, first to all the rows, and then to all the columns of the result, will produce a 1-scale wavelet transform.

We might hope that since the scaling and wavelet functions $\varphi(x)$ and $\psi(x)$ have such simple forms for the Haar wavelet, as we saw in Figure 15.3, the same would be true for the Daubechies-4 wavelet. We can plot a wavelet by inverting a “basis vector”: a vector consisting of all zeros except for a single one. Following [24] we shall choose a vector whose sixth element is one. We shall experiment first with the Haar wavelet:

```
>> [h,g] = daubcwf(2);
>> t = zeros(1,512); t(6) = 1;
>> td = miwt(t,h,9);
>> plot([1:512]/512,td,"linewidth",2)
```

MATLAB/Octave

The result is shown in Figure 15.10. Other basis vectors produce similar results, but with the wavelet scaled or translated. The vector we have chosen seems to produce the best plot. For the Daubechies-4 wavelet, we repeat the above four commands, but starting with

```
>> [h,g] = daubcwf(4);
```

MATLAB/Octave

In Python, the plot can be generated with

```
In : t = np.array([0]*512); t[5]=1  
In : td = rwt.idwt(t,h,9)[0]  
In : plt.plot(np.arange(512),td)
```

Python

The result is shown in Figure 15.11. This is a very spiky and unusual graph! It is in fact a fractal, and unlike the Haar wavelet, the function cannot be described in simple terms.

Figure 15.10: A plot of the Haar wavelet

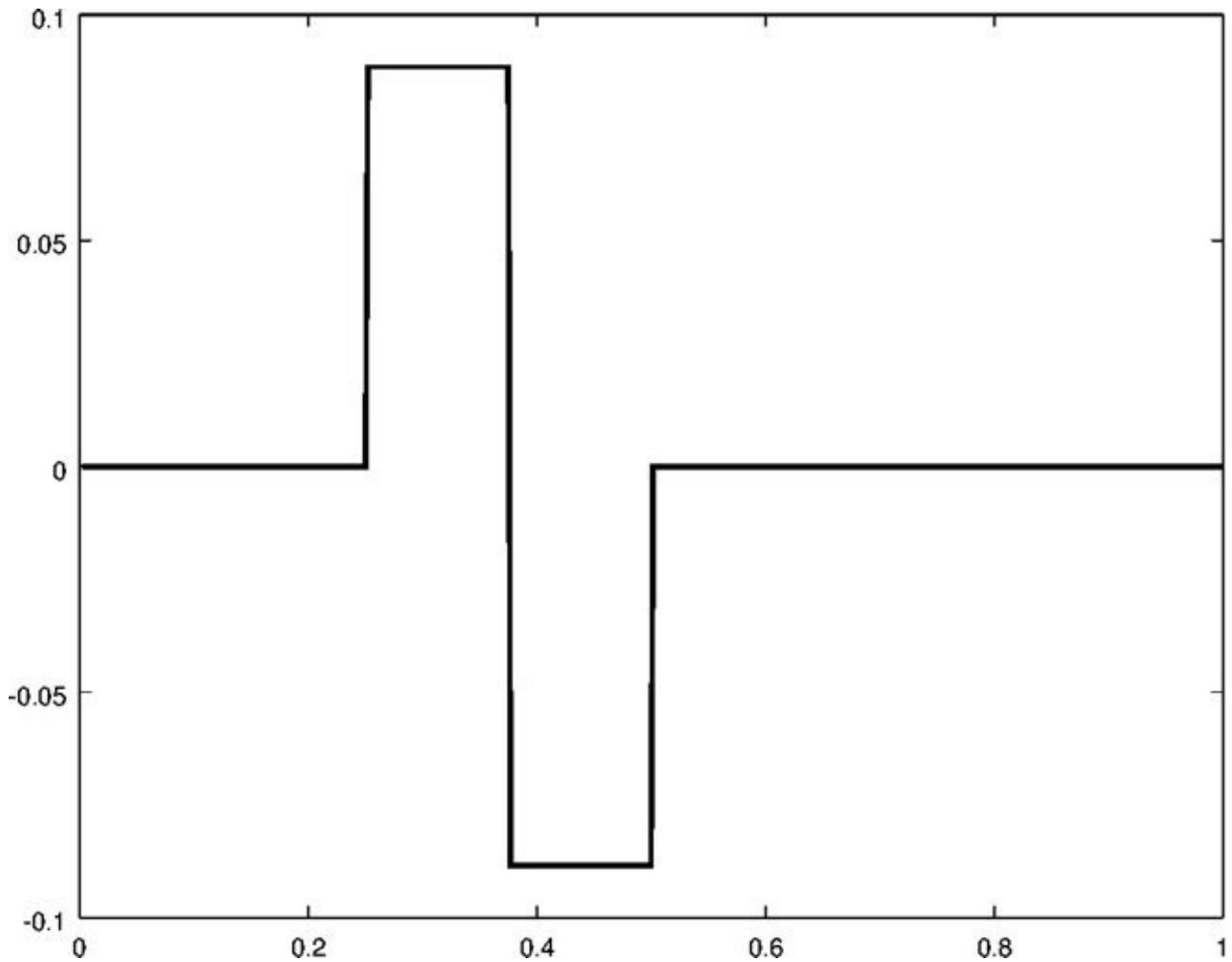
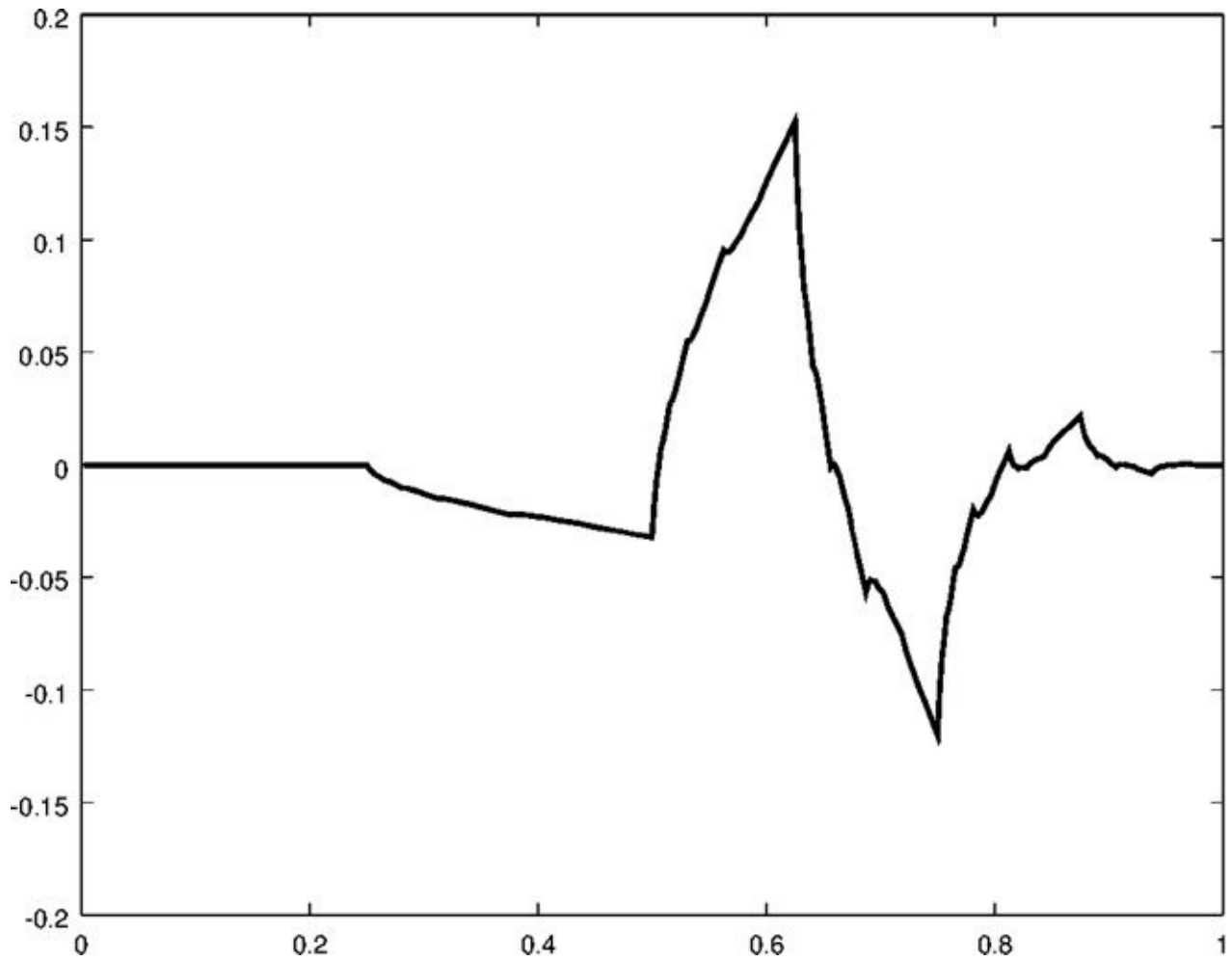


Figure 15.11: A plot of the Daubechies-4 wavelet



15.5 Image Compression Using Wavelets

Wavelets provide some of the most powerful tools known for image compression. As we stated earlier, they have replaced the use of the DCT in the JPEG2000 algorithm. In this section we shall play around with investigating what information we can remove from the DWT of an image, and still retain most of the original information.

Thresholding and Quantization

The idea is to take the DWT of an image, and for a given value d , set all values x in the DWT for which $|x| \leq d$ to zero. This is at the basis of the JPEG2000 compression [51]. We shall try this using the Daubechies-4 wavelets, and $d = 10$.

```
>> c = imread('caribou.png');
>> imshow(c);
>> cw = mdwt(double(c),h,8);
>> length(find(abs(cw)<=10))
```

```
ans =
```

```
47427
```

```
>> cw(find(abs(cw)<=10)) = 0;
>> cw = round(cw);
>> ci=midwt(cw,h,8);
>> imshow(mat2gray(ci))
```

MATLAB/Octave

We are thus removing nearly three quarters of the information from the DWT. We further simplify the result by quantizing; in this case we just use the `round` function to turn fractions into integers. The original caribou image and the result of the above commands are shown in Figure 15.12. The result is indistinguishable from the original image. We

can use different wavelets, by using either `daub(2)` or `daub(6)`, to obtain the Haar filter coefficients or the filter coefficients of the Daubechies-6 wavelet. The results of thresholding (again with $d = 10$) are shown in Figures 15.13(a) and (b). We could clearly obtain higher compression rates by removing more information; this can be obtained by increasing the threshold value d . We will try with $d = 30$ and $d = 50$, and using the Daubechies-4 wavelet:

Figure 15.12: An image after DWT thresholding and inversion



Figure 15.13: Use of different wavelets



(a) Using the Haar wavelet



(b) Using the Daubechies-6 wavelet

```
In : cw = mdwt(c.astype(double),h,8)
In : len(np.where(abs(cw)<=30)[0])
Out: 61093
In : cw[np.where(abs(cw)<=30)] = 0
In : cw = round(cw);
In : ci = rwt.idwt(cw,h,8);
In : io.imshow(ci)
In : cw = rwt.dwt(c.astype(double),h,8)
In : length(find(abs(cw)<=50))
Out: 63411
In : cw[np.where(abs(cw)<=50)] = 0
In : cw = np.round(cw);
In : ci = rwt.idwt(cw,h,8);
In : io.imshow(ci)
```

Python

The results are shown in Figure 15.14. Notice that with a threshold value of 50 we are throwing away 63,411 items of information, leaving only 2125 non-zero values. However, the result is extremely good when compared with the results of JPEG compression as shown in Figure 14.9 in Chapter 14.

Figure 15.14: Results of wavelet compression with $d = 30$ and $d = 50$



As we did in Chapter 14, we shall investigate the appearance of the image with a closeup. Figure 15.15 shows the closeup after thresholding at $d = 10$ using both the Haar wavelets and the Daubechies-6 wavelets. Figure 15.16 shows the closeup after thresholding at $d = 30$ and $d = 50$ using the Daubechies-6 wavelets. Even though we can see that we are losing some fine detail, the result is far superior to the result obtained by using the JPEG compression algorithm.

Extraction

Here we cut off a portion of the DWT, setting all values to zero, and invert the rest. This is not a standard method of compression, but it does give reasonably good results. It also has the advantage of being computationally easier than thresholding and quantization. For example, suppose we keep just the upper left 100×100 elements of the transform:

Figure 15.15: Closeups of wavelet compression using $d = 10$



Figure 15.16: Closeups of wavelet compression using $d = 30$ and $d = 50$



```
>> [h,g] = daubcwf(4);  
>> cw = mdwt(double(c),h,8);  
>> temp = zeros(size(c));  
>> temp(1:100,1:100) = cw(1:100,1:100);  
>> ci = midwt(temp,h,8);  
>> imshow(mat2gray(ci))
```

MATLAB/Octave

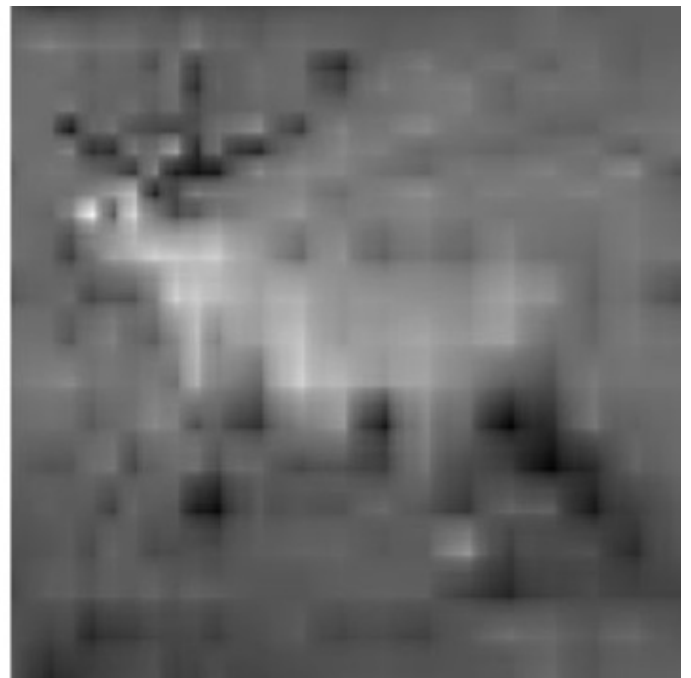
and the result is shown in Figure 15.17. Notice that whereas we took only 10,000 elements of the 65,536 elements of the transform—less than one-sixth—the output is remarkably good. Figure 15.18 shows the results after two more extractions. On the left, we used the top left 50×50 elements of the DWT; on the

right only 20×20 elements. The left-hand figure is getting quite blurry: but as we have used only 2500 of the possible 65,536 elements of the transform, this is to be expected. On the right, with only 400 elements used, the figure is basically unrecognizable. Even so, we can still distinguish size, shapes, and grayscales.

Figure 15.17: An image after DWT extraction and inversion



Figure 15.18: Compression by DWT extraction



15.6 High Pass Filtering Using Wavelets

If we look at a wavelet transform, then apart from the rescaled image in the top left, the rest is high frequency information. We would expect then, that if we were to eliminate the top left corner, by setting all the transform values to zero, the result after inversion would be a high pass filtered version of the original image.

We will perform a 2-scale decomposition (with Daubechies-4) on the caribou image, and remove the image from the top left.

```
>> cw = mdwt(double(c),h,2);  
>> cw(1:64,1:64) = 0;  
>> ci = midwt(cw,h,2);  
>> imshow(mat2gray(ci))
```

MATLAB/Octave

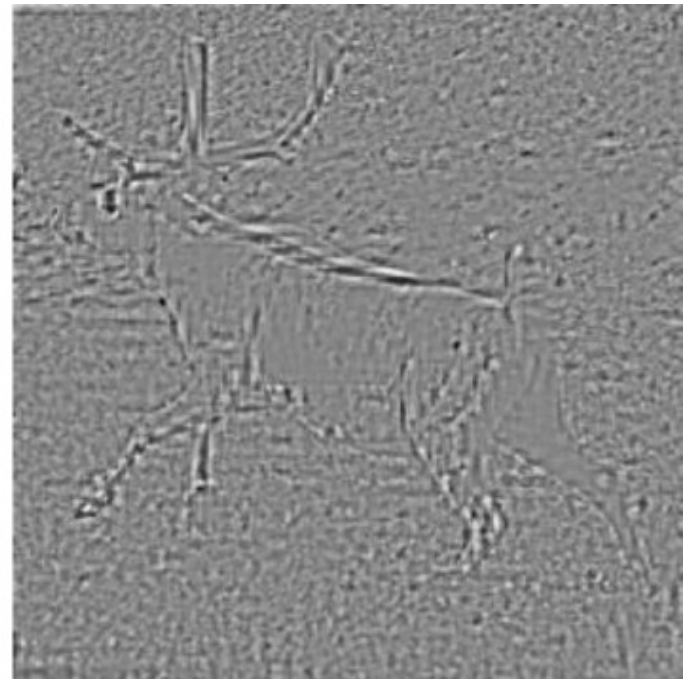
This is shown on the left in Figure 15.19. We can do the same thing but using a 3-scale decomposition; this will mean that the corner image will be half the size.

```
In : cw = rwt.dwt(c.astype(float),h,3)[0]  
In : cw[0:64,0:64] = 0;  
In : ci = rwt.idwt(cw,h,3);  
In : io.imshow(ci)
```

Python

This is shown on the right in Figure 15.19. These images can be thresholded to produce edge images.

Figure 15.19: High pass filtering with wavelets



We can be more specific in the part of the transform we eliminate or keep, and we can extract the horizontal or vertical edges only.

15.7 Denoising Using Wavelets

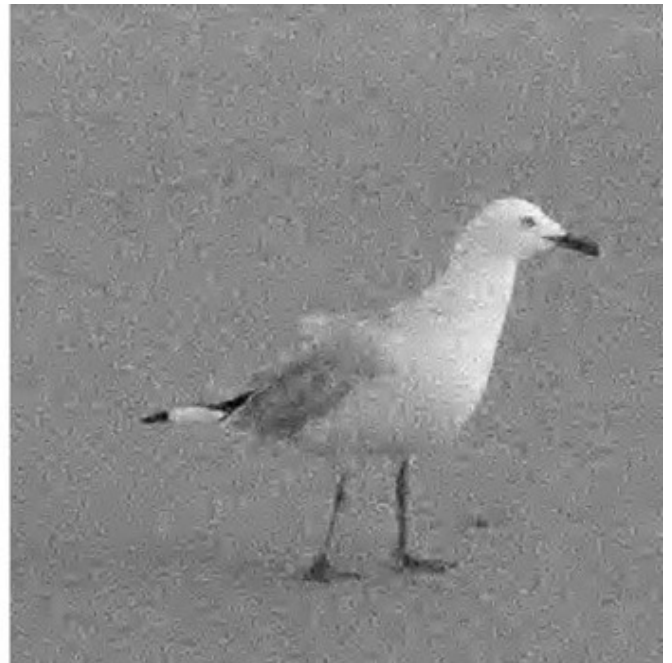
Since noise can be considered a high frequency component, some form of low pass filtering should remove it. As we did for compression, we can remove such components by thresholding. To compare with the results obtained in Chapter 8, we shall use a gray version of the seagull image, converted to type `double`.

```
>> f = im2double(rgb2gray(imread('gull.png')));
>> fg = imnoise(f,'gaussian');
>> imshow(fg)
```

MATLAB/Octave

Now we can attempt some noise removal. We shall assume that all the filter coefficients are those of the Daubechies-4 wavelet.

Figure 15.20: Denoising using wavelets



```
>> fw = mdwt(fg,h,4);
>> length(find(abs(fw)<=0.2))
ans =

    159192
```

MATLAB/Octave

Note that the image is of size $400 \times 400 = 160,000$, so we are looking at setting over 94% of the transform values to zero.

```
>> fw(find(abs(fw)<=0.2))=0;
>> fi = midwt(fw,h,4);
>> figure,imshow(mat2gray(fi))
```

MATLAB/Octave

The noisy image and the result after wavelet filtering are shown in Figure 15.20. At this level of thresholding, not much of the noise has been removed, although there has been some slight reduction. Thresholding can of course be done at higher levels:

```

In : f = ut.img_as_float(io.imread('fledgling.png'))
In : fg = ut.random_noise(f,'gaussian')
In : fw = rwt.dwt(fg,h,4)[0]
In : fw[np.where(abs(fw)<=0.4)] = 0
In : fi = rwt.idwt(fw,h,4)
In : io.imshow(fi[0])

```

Python

Figure 15.21 shows the results using (a) a threshold of 0.3 and (b) a threshold of 0.5. The results compare very favorably to those we investigated in Chapter 8; however, we can see that too high a threshold results in unacceptable blurring of the image.

Figure 15.21: Denoising using different thresholds



(a)



(b)

Exercises

- For each of the sequences
 - 20 38 6 25 30 29 21 32
 - 30 11 0 38 0 15 22 32
 - 7 38 19 11 24 14 32 14
 apply, by hand, the simple wavelet transform discussed in Section 15.1 at 1, 2, and 3 scales.
- Check your answers using the filter coefficients $h = [1, 1]$.
- Work backward from the transforms and obtain the original sequence.
- Repeat the above calculations in your system using the Haar wavelet transform; obtaining the filter coefficients with the command `daubcwf(2)`.
- For an 8-element vector v , determine its DWT at 1 scale using the Daubechies-4 wavelet. For the filter values h and g , determine the correlations with

```
>> v = double(v)
>> imfilter(v,h,'circular')
>> imfilter(v,g,'circular')
```

MATLAB/Octave

or

```
In : v = np.array(v).astype(float)
In : ndi.correlate(v,h,mode='wrap')
In : ndi.correlate(v,g,mode='wrap')
```

Python

Can you see how the wavelet is related to the results of the correlations? Write a program to perform the DWT at 1 scale using the Daubechies-4 wavelet by filtering. Then generalize your program to higher scales.

6. Obtain a 256×256 grayscale image. Obtain the DWT at 1, 2, and 3 scales using `daubc4f(2)`, `daubc4f(4)`, and `daubc4f(6)`. Are there any observable differences in the outputs of the transform for the different wavelets?
7. Write a function `showdwt` that will scale the result of a two-dimensional wavelet transform for easier viewing by scaling the individual square components of the DWT separately.
8. Choose any 256×256 grayscale image: Perform the forward DWT at 8 scales using the Daubechies-4 wavelet. For each of the following values of d :

- (a) 10,
- (b) 25,
- (c) 50,
- (d) 60

- Make all elements of the transform whose absolute values are less than d equal to zero.
- Determine the number of zeros in the result.
- Quantize the result by using the `floor` or `round` functions.
- Invert the result and display.

At what values of d do you notice degradation in the result?

9. Repeat the above question for some very large values of d : 100, 200, 300, ... What size of d renders the image completely unrecognizable? How many zeros are in the transform?
10. Repeat the above two questions with an image of a face: take a selfie, or a photograph of somebody else, and reduce it to a square image whose rows and columns are a power of 2.
11. Choose any of your 256×256 grayscale images. Add Gaussian noise with mean 0, and with standard variations 0.01, 0.02, 0.05, and 0.1. For each noisy image, attempt to denoise it with wavelets. How reasonable a result can you obtain for the noise with the higher standard deviations?